

El Viaje del Dato: Un Proceso ETL de Vectores para Azure

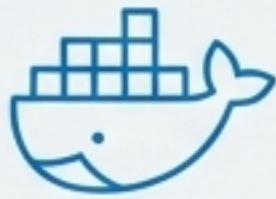
Desde la creación local hasta el consumo por Inteligencia Artificial en la nube.

Arquitectura del Flujo de Datos de Extremo a Extremo



Fase 1: El Origen de los Datos en un Entorno Local

Objetivo: Crear una base de datos vectorial para almacenar representaciones numéricas (embeddings) de imágenes del dataset FER2013.



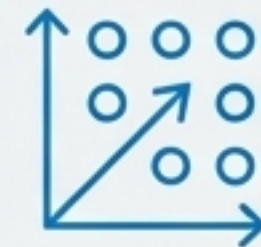
Contenedor Docker

Se utiliza una imagen ``ankane/pgvector`` para desplegar un servidor PostgreSQL con la extensión ``pgvector`` preinstalada, facilitando un entorno de desarrollo rápido y aislado.



Base de Datos

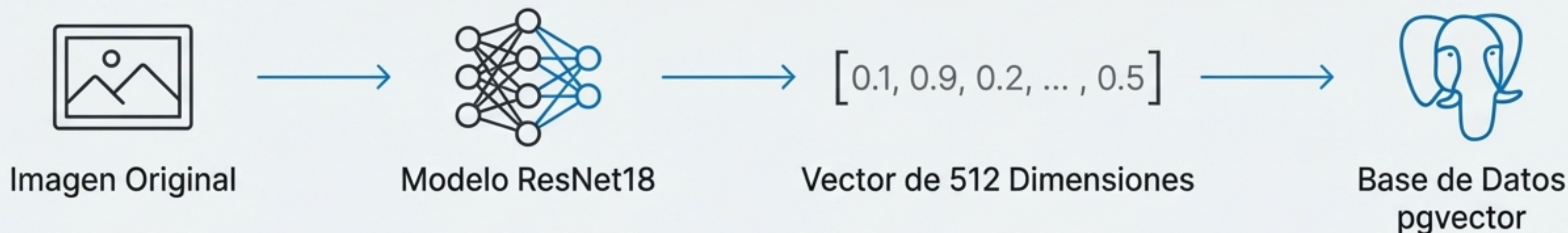
Se crea una base de datos llamada ``fer_vct``.



Extensión Vectorial

Se activa la extensión ``vector`` con el comando ``CREATE EXTENSION vector;``, habilitando el almacenamiento y la consulta de datos vectoriales de alta dimensionalidad.

Transformación de Imágenes a Vectores Numéricos



Proceso de Carga: La base de datos no almacena las imágenes, sino sus 'embeddings' vectoriales. Este proceso es gestionado por un script de Python (``cargar_fer.py``).

1. **Extracción de Características:** Utiliza un modelo de Deep Learning pre-entrenado (ResNet18) para analizar cada imagen y extraer sus características visuales.
2. **Conversión a Embedding:** Convierte estas características en un vector numérico de 512 dimensiones, una representación matemática de la imagen.
3. **Inserción en la Base de Datos:** Usa la librería ``psycopg2`` para conectar con la base de datos PostgreSQL local e insertar cada vector en la tabla ``imagenes_fer``, que contiene una columna de tipo ``VECTOR(512)``.

Fase 2: La Migración de los Datos a la Nube

Objetivo:

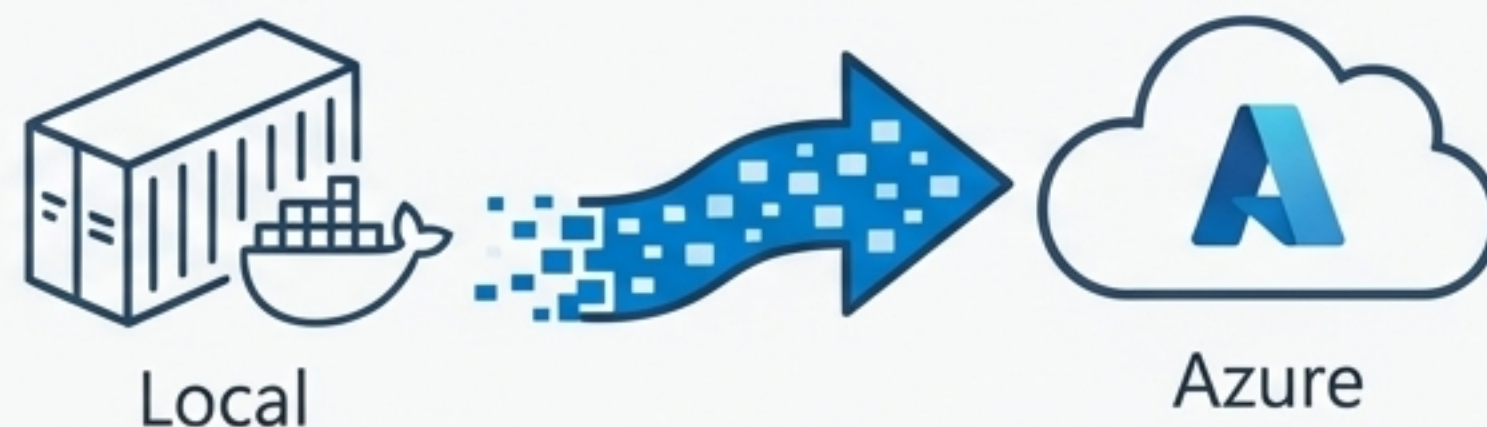
Trasladar la base de datos vectorial `fer\_vct` desde el contenedor Docker local a un servicio gestionado y escalable en Azure.

Servicio de Destino:

Azure Database for PostgreSQL - Flexible Server. Ofrece alta disponibilidad, escalabilidad y seguridad para cargas de trabajo de PostgreSQL en la nube.

Método:

Se utilizan las herramientas nativas de PostgreSQL, `pg\_dump` y `pg\_restore`, garantizando una migración de alta fidelidad y sin pérdida de datos.



Proceso de Migración y Configuración en Azure

Paso 1: Exportación Local (`pg\_dump`)



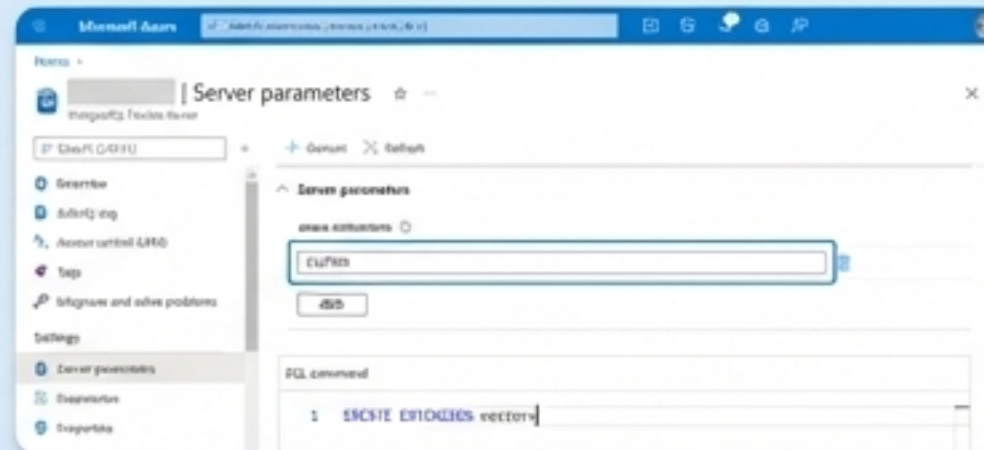
Se genera un archivo de respaldo (`.dump`) de la base de datos local.

```
pg_dump -h localhost -p 5433 -U postgres -Fc -d fer_vct > fer_vct_backup.dump
```

Paso 2: Habilitación de Extensión en Azure (Paso Crítico)



En el portal de Azure, dentro de los 'Parámetros del servidor' de PostgreSQL Flexible Server, se debe añadir 'VECTOR' a la lista de `azure.extensions`. Posteriormente, conectar al servidor y ejecutar `CREATE EXTENSION vector;`. Este paso autoriza el uso de la extensión en el entorno de Azure.



Paso 3: Importación en Azure (`pg\_restore`)



Se utiliza el archivo de respaldo para restaurar la base de datos directamente en la instancia de Azure PostgreSQL.

```
pg_restore -h <AZURE_HOST_NAME> ... -d <DB_NAME> "<RUTA_AL_DUMP>"
```

****Nota de Verificación:**** Tras la migración, se confirma la integridad de los datos verificando el conteo de filas en la tabla `imagenes\_fer` en Azure.

Fase 3: Orquestación y Procesamiento con Azure Data Factory (ADF)

Rol de ADF: Actúa como la plataforma central de ELT (Extract, Load, Transform) nativa de la nube. Su función es orquestar el movimiento de los datos desde Azure PostgreSQL y Azure Blob Storage hacia el destino final: un Data Lakehouse.



Paso Previo Manual: Las imágenes originales del dataset (archivos .png) deben ser subidas a un contenedor en Azure Blob Storage. Esto se puede realizar con herramientas como Azure Storage Explorer o la CLI de Azure. Una vez en la nube, todo el flujo de datos es gestionado por servicios de Azure.

La Base del Análisis: Construcción del Data Lakehouse

Tecnología: Azure Data Lake Storage Gen2 (ADLS Gen2).

Característica Esencial: Namespace Jerárquico

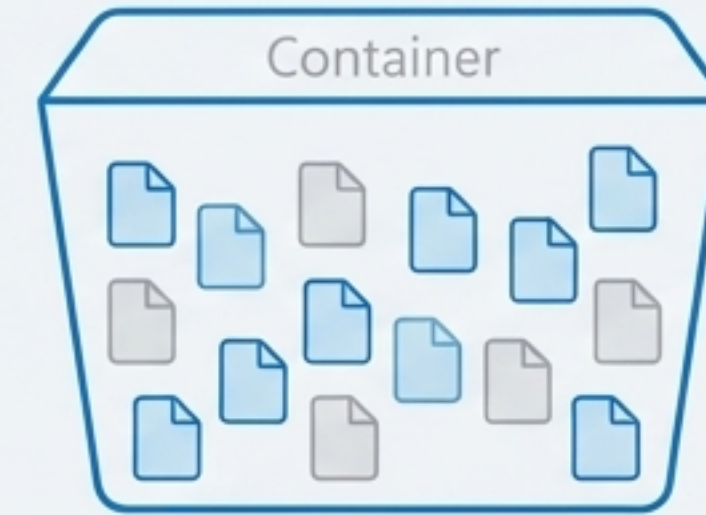
Durante la creación de la Storage Account, se habilita la opción 'Hierarchical namespace'.

Esta configuración transforma el almacenamiento de objetos en un verdadero sistema de archivos, permitiendo la gestión eficiente de directorios y archivos. Es lo que define a ADLS Gen2 como un Data Lakehouse, optimizado para cargas de trabajo de análisis a gran escala.

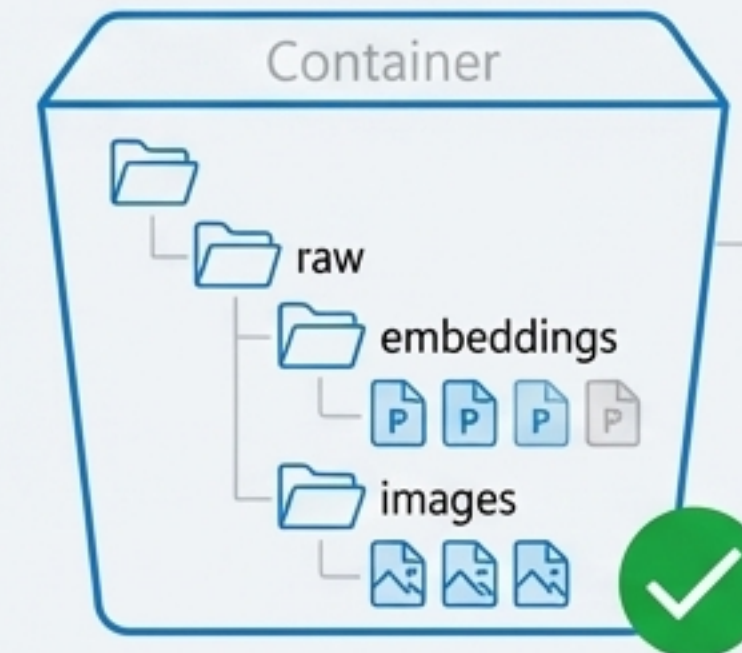
Estructura de Datos:

- Los embeddings vectoriales se almacenarán en formato **Parquet**: un formato columnar eficiente, optimizado para consultas analíticas.
- Las imágenes originales se almacenarán como archivos binarios.
- Ubicación de ejemplo: `/raw/embeddings/fer2013/` y `/raw/images/fer2013/`.

Almacenamiento de Objetos (Plano)



Namespace Jerárquico (ADLS Gen2)



Configuración de Conexiones y Conjuntos de Datos en ADF

Linked Services (Las Conexiones)

Son las cadenas de conexión que ADF utiliza para comunicarse con los almacenes de datos externos.



Azure PostgreSQL:
Para leer los embeddings
de la tabla
'imagenes_fer'.



**Azure Data Lake Storage
Gen2:** Para escribir los
archivos Parquet
(embeddings) y los archivos
binarios (imágenes).



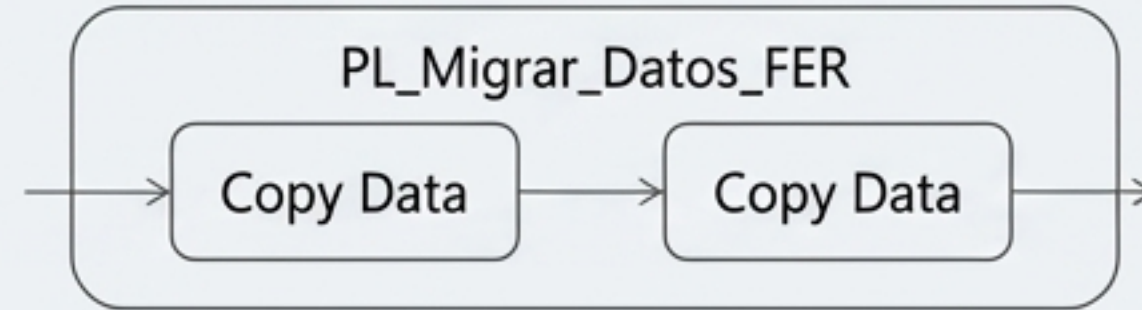
Azure Blob Storage: Para leer
los archivos de imagen
originales que fueron
subidos previamente.

Datasets (Los Punteros a Datos)

Definen la estructura y ubicación específica de los datos dentro de un Linked Service.

- 'DS_PgVector_ImagenesFer': Apunta a la tabla de embeddings en PostgreSQL.
- 'DS_Lakehouse_Embeddings': Define el archivo de salida en formato Parquet en ADLS Gen2.
- 'DS_Images_Source': Apunta a los archivos de imagen en formato binario en Blob Storage.
- 'DS_Lakehouse_Images': Define la ubicación de salida para las imágenes en ADLS Gen2.

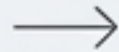
El Pipeline en Acción: Dos Flujos de Datos en Paralelo



Flujo 1: Migración de Embeddings (PostgreSQL -> Parquet)



Activity: `Copy\_Embeddings\_PostgreSQL`



Origen:

Utiliza una consulta SQL en lugar de una tabla directa.



Transformación Clave:

La consulta modifica la ruta del archivo (`filepath`) al vuelo para que coincida con la nueva ubicación de las imágenes en el Data Lakehouse.

```
SELECT id, emotion, vector,  
       'raw/imagenes/train/' || filepath AS filepath  
FROM public.imagenes_fer;
```



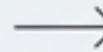
Destino:

Escribe los datos, incluyendo el `filepath` corregido, en un archivo Parquet en ADLS Gen2.

Flujo 2: Migración de Imágenes (Blob Storage -> ADLS Gen2)



Activity: `Copy\_Images\_BlobStorage`



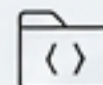
Origen:

Lee los archivos de imagen binarios desde Azure Blob Storage.



Comportamiento de Copia:

Se configura con la opción '**Preserve hierarchy**'. Esto es crucial para mantener la estructura de carpetas original (ej. `train/happy/`), asegurando que las rutas corregidas en el flujo de embeddings sean válidas.



Destino:

Copia los archivos de imagen sin alteración al Data Lakehouse.

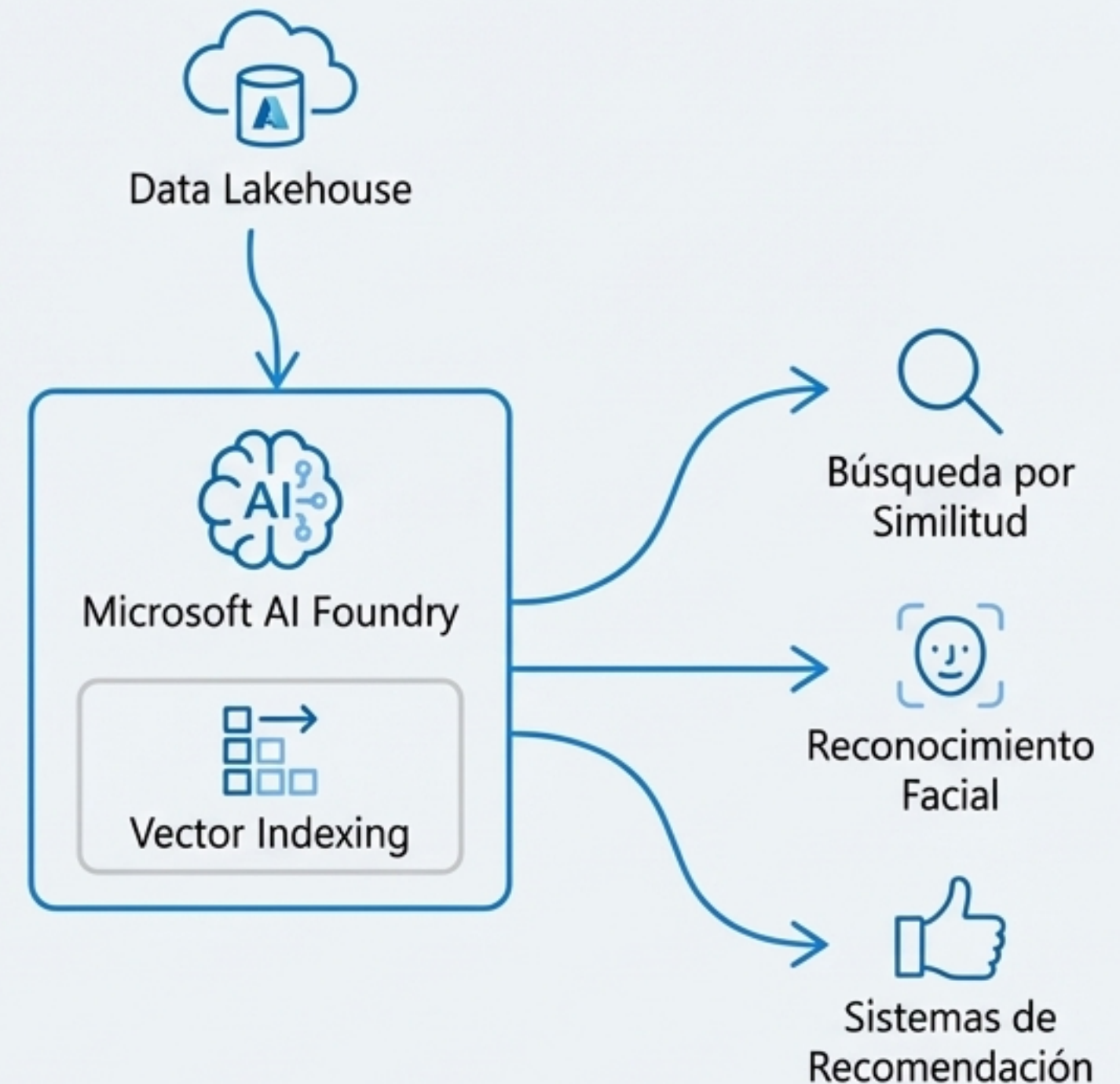
Fase 4: El Destino de los Datos - Consumo con Inteligencia Artificial

Objetivo Final: Utilizar los embeddings vectoriales y las imágenes, ahora organizados en el Data Lakehouse, para construir y potenciar aplicaciones de IA.

Plataforma de Consumo: Microsoft AI Foundry (o servicios equivalentes de Azure AI).

Proceso de Consumo:

1. **Acceso a Datos:** El servicio de IA se configura con acceso de lectura directo al Data Lakehouse (ADLS Gen2).
2. **Indexación Vectorial:** Los vectores se leen desde los archivos Parquet y se cargan en un índice vectorial de alto rendimiento.
3. **Habilitación de Consultas:** Este índice permite realizar búsquedas de similitud ultrarrápidas, la base para aplicaciones como búsqueda de imágenes similares, reconocimiento facial, y sistemas de recomendación.



Del Entorno Local a la Inferencia IA: Un Flujo de Datos Completo

****Resumen del Viaje:**** Hemos seguido el dato desde su creación hasta su aplicación final, construyendo un pipeline robusto y escalable en Azure.



Este proceso demuestra cómo la arquitectura moderna de datos en la nube transforma datos brutos en un activo estratégico para la Inteligencia Artificial.